BÀI 1

TỔNG QUAN VỀ ĐẢM BẢO CHẤT LƯỢNG PHẦN MỀM

MỤC LỤC

1.1. SẢN XUẤT PHẦN MỀM VÀ THÁCH THỨC VỀ CHẤT LƯỢNG PHẦN MỀM 3
1.1.1. Phần mềm 3
1.1.2. Sản xuất phần mềm 5
1.1.3. Quy trình phát triển phần mềm 6
1.1.4. Thách thức đảm bảo chất lượng phần mềm 8
1.2. CHẤT LƯỢNG VÀ ĐẢM BẢO CHẤT LƯỢNG 11
1.2.1. Độ tin cậy phần mềm 11
1.2.2. Lỗi phần mềm và nguyên nhân 11
1.2.3. Chất lượng phần mềm 14
1.2.4. Đảm bảo chất lượng phần mềm 17
1.3. CÁC YẾU TỐ ẢNH HƯỞNG TỚI HOẠT ĐỘNG ĐẢM BẢO CHẤT LƯỢNG PHẦN MỀM 18
1.3.1. YẾU TỐ CON NGƯỜI 18
1.3.1.1. Đội dự án 18
1.3.1.2. Thành viên ngoài 20
1.3.2. Yếu tố dự án 20
1.3.3. Yếu tố quy trình 21
1.3.3.1. Quy trình phát triển phần mềm 22
1.3.3.2. Quy trình kiểm thử phần mềm 22
1.3.3.3. Quy trình quản lý cấu hình phần mềm 23
1.3.3.4. Quy trình quản lý thay đổi 24
1.3.4. Yếu tố công nghệ 24
1.3.4.1. Môi trường phát triển 25
1.3.4.2. Công nghệ phát triển và quản lý phần mềm 26
1.3.5. Yếu tố đánh giá và đo lường 27

1.3.5.1. Thiết lập tiêu chuẩn đánh giá chất lượng 28
1.3.5.2. Xác định độ đo 28
1.3.5.3. Phương pháp đánh giá chất lượng phần mềm 29
1.3.6. Yếu tố quản lý 29
1.4. CÁC THÀNH PHẦN ĐẢM BẢO CHẤT LƯỢNG PHẦN MỀM 30
1.4.1. Công cụ hỗ trợ đảm bảo chất lượng phần mềm 30
1.4.1.1. Thủ tục và chỉ dẫn 30
1.4.1.2. Mẫu và danh mục kiểm tra 32
1.4.2. Kiểm soát tiến độ 34
1.4.2.1. Vai trò của kiểm soát tiến độ dự án đến SQA 34
1.4.2.2. Các hoạt động kiểm soát 35
1.4.2.3. Phân bổ trách nhiệm thực hiện các nhiệm vụ kiểm soát 37
1.5. CÂU HỎI VÀ BÀI TẬP 38
1.5.1. Câu hỏi 38
1.5.2. Bài tập 38

BÀI 1

TỔNG QUAN VỀ ĐẢM BẢO CHẤT LƯỢNG PHẦN MỀM

1.1. SẢN XUẤT PHẦN MỀM VÀ THÁCH THỨC VỀ CHẤT LƯỢNG PHẦN MỀM

1.1.1. Phần mềm

1.1.1.1. Khái niệm

Phần mềm dưới góc nhìn của người sử dụng:

- Chương trình thực thi được trên máy tính hoặc các thiết bị chuyên dụng khác;
- Nhằm hỗ trợ cho các nhà chuyên môn trong từng lĩnh vực chuyên ngành thực hiện tốt hơn các thao tác nghiệp vụ của mình.

Phần mềm dưới góc nhìn của chuyên viên Tin học: đây là một hệ thống bao gồm 3 thành phần cơ bản:

- Thành phần giao tiếp;
- Thành phần xử lý;
- Thành phần lưu trữ.

cần được xây dựng để thực hiện theo yêu cầu của người sử dụng.

Vậy, có thể hiểu: Phần mềm là một tập quy tắc xử lý thể hiện thành chương trình (mã lệnh+dữ liệu) được cài đặt vào phần cứng phù hợp để tự thực hiện một vài công việc thay con người. Các mô tả cho chương trình (chức năng, giao diện,cách sử dụng, ràng buộc,..), để nhiều người cùng hợp tác với nhau làm ra & sử dụng phần mềm: phân tích viên, thiết kế viên, lập trình viên, kiểm thử viên, người sử dụng, admin,...

1.1.1.2. Định nghĩa IEEE về phần mềm

Phần mềm bao gồm các thành phần:

- Chương trình máy tính (code);
- Các thủ tục;

- Tài liệu;
- Dữ liệu cần thiết cho sự vận hành của hệ thống.

Trong đó:

- Chương trình máy tính (code): giúp máy tính vận hành thực thi các yêu cầu.
- Thủ tục: được yêu cầu để định nghĩa theo một thứ tự và lịch biểu của một chương trình khi thực thi, phương thức được triển khai và chịu trách nghiệm cho thực thi các hoạt động cần thiết cho việc tác động vào phần mềm.
- Tài liệu:
    + Cần thiết cho người phát triển, sử dụng và bảo trì.
    + Cho phép sự phối hợp và cộng tác giữa các thành viên trong đội ngũ phát triển; rà soát các sản phẩm lập trình và thiết kế.
    + Cung cấp một sự miêu tả cho ứng dụng và những phương pháp thích hợp cho việc sử dụng.
    + Cung cấp cho đội bảo trì tất cả những thông tin yêu cầu về mã nguồn và công việc và cấu trúc cho từng module.
- Dữ liệu cần thiết cho sự vận hành của hệ thống:
    + Dữ liệu bao gồm các tham số đầu vào, mã nguồn và danh sách tên thích hợp với phần mềm để đặc tả những cái cần thiết cho người sử dụng thao tác với hệ thống.
    + Dữ liệu cần thiết là chuẩn dữ liệu test.

1.1.1.3. Đặc điểm của phần mềm

Phần mềm có các đặc điểm sau:

- Không có tính chất vật lý (vô hình);
- Không bị hao mòn như phần cứng, chỉ bị lạc hậu;
- Sao chép được;

- Sự thay đổi linh hoạt là ưu thế của phần mềm so với phần cứng. Do đó, cách làm ra phần mềm cũng khác:
    + Dựa trên sự tư duy để sáng tác ra phần mềm
    + Phần mềm được sử dụng qua các versions

1.1.2. Sản xuất phần mềm

Lĩnh vực sản xuất phần mềm hay còn gọi là **Công nghệ phần mềm** là lĩnh vực khoa học về các phương pháp luận, kỹ thuật và công cụ tích hợp trong quy trình sản xuất và vận hành phần mềm. Công nghệ phần mềm là một ngành kỹ thuật liên quan đến tất cả các khía cạnh của sản xuất phần mềm từ giai đoạn đầu của đặc tả hệ thống cho đến bảo trì hệ thống. Sản xuất phần mềm không chỉ là quá trình phát triển kỹ thuật, mà còn bao gồm công tác quản lý dự án, phát triển các công cụ, phương pháp, tài liệu, v.v. để hỗ trợ sản xuất phần mềm.

Có nhiều định nghĩa về ngành công nghệ phần mềm, như:

Bauer (1969): việc thiết lập và sử dụng các nguyên tắc công nghệ học đúng đắn dùng để thu được phần mềm một cách kinh tế vừa tin cậy vừa làm việc hiệu quả trên các máy thực.

Parnas (1987): việc xây dựng phần mềm nhiều phiên bản bởi nhiều người.

Ghezzi (1991): một lĩnh vực của khoa học máy tính, liên quan đến xây dựng các hệ thống phần mềm vừa lớn vừa phức tạp bởi một hay một số nhóm kỹ sư.

IEEE (1993): nghiên cứu các phương pháp tiếp cận và việc áp dụng phương pháp tiếp cận có hệ thống, bài bản và được lượng hóa trong phát triển, vận hành và bảo trì phần mềm;

Pressman (1995): công nghệ phần mềm là bộ môn tích hợp cả quy trình, các phương pháp, các công cụ để phát triển phần mềm máy tính.

Từ các vấn đề trên cho thấy, sản phẩm phần mềm cũng là một sản phẩm sản xuất công nghiệp. Tuy nhiên, phần mềm là một sản phẩm đặc biệt và không giống như các sản phẩm công nghiệp khác. Quy trình sản xuất phần mềm hoàn toàn khác

biệt so với các sản phẩm sản xuất công nghiệp khác. Do vậy, người ta gọi quá trình sản xuất phần mềm là "phát triển phần mềm" thay cho "sản xuất phần mềm".

Sản phẩm phần mềm có các đặc điểm sau:

- Không có tính chất vật lý;
- Không bị hao mòn;
- Sao chép được;
- Sự thay đổi linh hoạt.

Trong đó, sự thay đổi linh hoạt là ưu thế của phần mềm so với đa số các sản phẩm công nghiệp khác. Do đó, cách làm ra phần mềm cũng khác.

1.1.3. Quy trình phát triển phần mềm

Vòng đời phát triển phần mềm (SDLC-Software Development Life Cycle) là một quá trình bao gồm một loạt các hoạt động được lên kế hoạch để phát triển hoặc thay đổi sản phẩm phần mềm.

SDLC bao gồm một kế hoạch chi tiết mô tả cách phát triển, bảo trì, thay thế và thay đổi hoặc nâng cao phần mềm cụ thể. SDLC được ngành công nghiệp phần mềm vận dụng để phát triển hệ thống phần mềm chất lượng cao, đáp ứng hoặc vượt qua mong đợi của khách hàng, hoàn thành trong thời gian và chi phí ước tính ban đầu. SDLC xác định một phương pháp luận để cải thiện chất lượng của phần mềm và quá trình phát triển tổng thể.

SDLC bao gồm 3 giai đoạn: Giai đoạn định nghĩa, giai đoạn phát triển và giai đoạn hỗ trợ.

(i) Giai đoạn định nghĩa: Giai đoạn định nghĩa tập trung vào làm rõ các vấn đề cho câu hỏi **Cái gì?**

- Thông tin gì cần xử lý (đầu vào và đầu ra);
- + Các chức năng mà phần mềm cần có và thực hiện được;
- + Các hành vi của hệ thống sẽ được mong đợi;
- + Các tiêu chuẩn hợp lệ sử dụng để đánh giá được sự đúng đắn và thành

công của hệ thống;

(ii) Giai đoạn phát triển: tập trung vào vấn đề **Làm thế nào?**

- Kiến trúc hệ thống được tổ chức thế nào?
- Các chức năng được cài đặt và liên kết với nhau thế nào?
- Tổ chức các cấu trúc dữ liệu, cơ sở dữ liệu thế nào?
- Chuyển từ thiết kế sang cài đặt thế nào?
- Việc kiểm thử sẽ được thực hiện thế nào?

(iii) Giai đoạn hỗ trợ: còn gọi là giai đoạn bảo trì, tập trung vào việc ứng phó với các thay đổi của hệ thống phần mềm, bao gồm:

- Sửa lỗi (Correction);
- Làm thích ứng (Adaptation);
- Nâng cấp (Upgrade);
- Phòng ngừa (Prevention), còn gọi là tái kỹ thuật phần mềm (software reengineering)

Các hoạt động cơ bản của SDLC:

- Lập kế hoạch và phân tích yêu cầu (Planning and requirement analysis)
- Xác định/Định nghĩa yêu cầu (Defining requirements)
- Thiết kế kiến trúc sản phẩm (Designing the product architecture)
- Xây dựng/phát triển sản phẩm (Building/Developing the product)
- Kiểm thử sản phẩm (Testing the product)
- Triển khai/phát hành trên thị trường và bảo trì sản phẩm (Deployment in the market and maintenance).

Có thể hiểu quy trình phát triển phần mềm là một quá trình bao gồm một loạt các hoạt động được lên kế hoạch để phát triển hoặc thay đổi sản phẩm phần mềm. Mô hình quy trình (procedure model) là một chiến lược phát triển phần mềm. Khác với quy trình, mô hình quy trình bao gồm các cách thức kết hợp, sử dụng tiến trình (process) phần mềm, cách vận dụng các phương pháp và các công

cụ trong mỗi giai đoạn phát triển. Mỗi mô hình quy trình tuân theo một loạt các bước duy nhất cho kiểu của nó để đảm bảo sự thành công trong quá trình phát triển phần mềm.

Một số mô hình quy trình phổ biến: Waterfall, Iterative, Spiral, Big Bang, V-Model, mô hình Scrum, mô hình phát triển phần mềm thích ứng (ASD), mô hình phát triển hệ thống động (DSDM), mô hình phát triển theo hướng tính năng (FDD), ….

1.1.4. Thách thức đảm bảo chất lượng phần mềm

Sự phức tạp lớn cũng như khả năng vô hình của phần mềm, cùng với sự khác biệt về các đặc tính sản phẩm so với sản phẩm công nghiệp, làm cho việc phát triển phương pháp luận SQA và vận dụng triển khai thành công nó là một thách thức chuyên môn cao. Điều này được phản ánh từ sự khác biệt cơ bản trong quy trình sản xuất giữa phần mềm và các sản phẩm công nghiệp khác. Những khác biệt này có thể được nhìn nhận trên 3 khía cạnh: *(i) Độ phức tạp của sản phẩm; (ii) Khả năng hiển thị của sản phẩm và (iii) Quá trình sản xuất và phát triển sản phẩm.*

(i) Mức độ phức tạp: có thể đánh giá mức độ phức tạp của sản phẩm bằng cách đo lường bằng số lượng các chế độ hoạt động của sản phẩm. Đối với một sản phẩm công nghiệp, có thể đến hàng nghìn chế độ hoạt động, được tạo ra bởi sự kết hợp của các thiết lập vật lý và logic máy khác nhau. Tuy nhiên, với sản phẩm phần mềm thì khác. Với một gói phần mềm điển hình, người ta có thể tìm thấy hàng triệu hoặc nhiều hơn nữa khả năng vận hành, hoạt động được phát triển một cách chính xác. Đây chính là một thách thức lớn đối với ngành công nghiệp phần mềm.

(ii) Khả năng hiển thị trực quan: Đối với hầu hết các sản phẩm công nghiệp, người ta có thể nhìn thấy được hình dạng (bằng mắt thường hoặc các công cụ hỗ trợ khác). Trong khi đó, với sản phẩm phần mềm là vô hình (không có tính chất vật lý). Nhà sản xuất có thể phát hiện hầu hết các "lỗi" của một sản phẩm sản xuất

công nghiệp trong quá trình phát triển và sản xuất. Trong khi đó, với các khiếm khuyết trong sản phẩm phần mềm thì rất khó và gần như không thể để phát hiện.

(iii) Quá trình sản xuất và phát triển: Bảng 1.1 so sánh các giai đoạn trong sản xuất phần mềm và sản phẩm công nghiệp khác. Các giai đoạn bao gồm: *(1) Phát triển sản phẩm; (2) Lập kế hoạch sản xuất sản phẩm; (3) Sản xuất.*

Bảng 1.1. Các giai đoạn sản xuất và phát triển sản phẩm

Hoạt động	**Sản phẩm công nghiệp**	**Sản phẩm phần mềm**
Giai đoạn phát triển sản phẩm	Nhà chuyên môn sẽ kiểm tra và thử nghiệm nguyên mẫu sản phẩm để phát hiện ra các lỗi của sản phẩm.	Nhà phát triển và các chuyên gia đảm bảo chất lượng phần mềm được hướng tới việc phát hiện các lỗi cố hữu của sản phẩm.
Giai đoạn lập kế hoạch để sản xuất	Các công cụ được thiết kế và chuẩn bị hỗ trợ trong giai đoạn lập kế hoạch sản xuất. Đây cũng chính là giai đoạn giúp cho nhà phát triển có thêm cơ hội để kiểm tra sản phẩm, điều này có thể phát hiện ra những khiếm khuyết đã "thoát khỏi" các đánh giá và thử nghiệm được thực hiện trong giai đoạn phát triển.	Giai đoạn này không bắt buộc đối với quá trình sản xuất phần mềm. Nếu có, ở đây chỉ bao gồm quá trình nhân bản các **bản sao** (phần mềm và tài liệu hướng dẫn kỹ thuật).
Sản xuất	Ở giai đoạn này, các thủ tục QA được nhà phát triển sử dụng để phát hiện "lỗi" của chính sản phẩm. Các khiếm khuyết của sản phẩm được phát hiện trong giai đoạn này	Giai đoạn này cũng không có trong các hoạt động sản xuất phần mềm. Nhiệm vụ giai đoạn này chỉ là nhân bản sản phẩm và tài liệu kỹ thuật đi kèm sản

Hoạt động	Sản phẩm công nghiệp	Sản phẩm phần mềm
	thường có thể được sửa chữa (thay đổi thiết kế, thay đổi vật liệu hoặc công cụ sản xuất) theo cách loại bỏ những khiếm khuyết đó cho các sản phẩm.	phẩm. Do đó, kỳ vọng phát hiện các lỗi khá hạn chế trong giai đoạn này.

Thông tin trên Bảng 1.1. cho thấy khả năng phát hiện ra các lỗi trong một sản phẩm công nghiệp và sản phẩm phần mềm là hoàn toàn khác nhau. Quá trình sản xuất sản phẩm công nghiệp khác được thực hiện qua 3 giai đoạn độc lập, cho khả năng phát hiện lỗi sản phẩm ngay từ giai đoạn trước khi sản xuất. Trong khi đó, rất khó khăn để phát hiện ra lỗi sản phẩm phần mềm.

Việc sản xuất phần mềm chỉ bao gồm sao chép sản phẩm và tài liệu kỹ thuật phần mềm. Do đó, kỳ vọng phát hiện các lỗi khá hạn chế trong giai đoạn này. Sự khác biệt này ảnh hưởng đến việc phát hiện các lỗi của sản phẩm phần mềm so với việc tìm ra lỗi của sản phẩm công nghiệp khác. Bảng 1.2 chỉ ra các yếu tố ảnh hưởng đến việc phát hiện lỗi phần mềm so với sản phẩm sản xuất công nghiệp khác.

Bảng 1.2. Các yếu tố ảnh hưởng đến việc phát hiện lỗi trong sản phẩm phần mềm so với các sản phẩm công nghiệp khác

Đặc tính	Sản phẩm phần mềm	Sản phẩm công nghệ khác
Độ phức tạp của sản phẩm	Thông thường, sản phẩm rất phức tạp cho phép nhiều tùy chọn hoạt động	Mức độ phức tạp thấp hơn nhiều, cho phép tối đa vài nghìn tùy chọn hoạt động
Khả năng hiển thị của sản phẩm	Sản phẩm vô hình, không thể phát hiện ra các khiếm khuyết hoặc thiếu sót	Sản phẩm có thể nhìn thấy được, cho phép phát hiện hiệu quả các lỗi bằng mắt

Bản chất của sự phát triển	Cơ hội phát hiện các lỗi và quy trình sản xuất chỉ phát sinh trong một giai đoạn, đó là phát triển sản phẩm	Cơ hội phát hiện các lỗi phát sinh trong tất cả các giai đoạn phát triển và sản xuất: ■ Phát triển sản phẩm ■ Lập kế hoạch sản xuất sản phẩm ■ Sản xuất

1.2. CHẤT LƯỢNG VÀ ĐẢM BẢO CHẤT LƯỢNG

1.2.1. Độ tin cậy phần mềm

Độ tin cậy của phần mềm được đề cập đến khi sản xuất một sản phẩm phần mềm để hạn chế nhiều nhất có thể và giảm thiểu các lỗi người dùng gặp phải trong quá trình sử dụng phần mềm.

Độ tin cậy của phần mềm được định nghĩa là xác suất hoạt động không có lỗi của hệ thống phần mềm trong một thời gian xác định trong một môi trường cụ thể. Mức độ tin cậy của hệ thống phụ thuộc vào các yếu tố đầu vào gây ra lỗi mà người dùng cuối có thể quan sát được. Hamlet (2001) [1] chỉ ra độ tin cậy của phần mềm có thể được ước tính thông qua thử nghiệm ngẫu nhiên. Độ tin cậy là đặc trưng cho một "môi trường cụ thể", dữ liệu thử nghiệm phải được rút ra từ phân phối đầu vào sao cho gần giống với cách sử dụng hệ thống trong tương lai. Mô hình hoạt động và sử dụng phần mềm của người dùng trong tương lai của một hệ thống phần mềm theo nghĩa chung được John D. Musa (1996) [1] mô tả trong một biểu mẫu được gọi là **Hồ sơ hoạt động**.

1.2.2. Lỗi phần mềm và nguyên nhân

1.2.2.1. Lỗi phần mềm

Nguồn gốc của bất kì sự hỏng hóc phần mềm (Failure) nào cũng thường bắt đầu từ một sai sót (Error), và các sai sót sẽ gây ra các lỗi (Fault hay Defect) phần mềm. Có thể hiểu Error, Fault và Failure như sau:

- Error – Sai sót: là sự không nhất quán giữa giá trị đầu ra của phần mềm so với giá trị đúng tương ứng với một đầu vào. Một sai sót (error) là một sự nhầm lẫn hay một sự hiểu sai trong quá trình phát triển phần mềm của người phát triển

- Fault – Lỗi: một trạng thái là nguyên nhân làm cho hệ thống hỏng khi thực hiện chức năng nào đó. Lỗi xuất hiện trong phần mềm như là kết quả của một sai sót. Lỗi (fault, defect) xuất hiện trong phần mềm như là kết quả của một sai sót.

- Faulure – Hỏng: sự bất lực của phần mềm khi thực hiện một chức năng theo đặc tả. Một hỏng hóc (failure) là kết quả của một lỗi xuất hiện làm cho chương trình không hoạt động được hay hoạt ñộng nhưng cho kết quả không như mong đợi. Các faults trở thành failures chỉ khi chúng được "activated" đó là khi ng ười dùng cố gắng áp dụng các phần m ềm cụ thể đó bị faulty. Do đó, nguồn gốc của bất kì Failure nào là một Error.

1.2.2.2. Nguyên nhân gây lỗi phần mềm

Có 9 nguyên nhân gây lỗi phần mềm chính:

- Lỗi định nghĩa yêu cầu:
    + Sai sót trong định nghĩa các yêu cầu.
    + Không có các yêu cầu quan trọng.
    + Không hoàn chỉnh định nghĩa các yêu cầu.
    + Bao gồm các yêu cầu không cần thiết, các chức năng mà không thực sự cần thiết trong tương lai gần.
- Lỗi giao tiếp giữa khách hàng và người phát triển:
    + Hiểu sai các chỉ dẫn của khách hàng như đã nêu trong các tài liệu yêu cầu.
    + Hiểu sai các yêu cầu thay đổi của khách hàng được trình bày với nhà phát triển bằng văn bản trong giai đoạn phát triển.
    + Hiểu sai của các yêu cầu thay đổi của khách hàng được trình bày bằng lời nói với nhà phát triển trong giai đoạn phát triển.

+ Hiểu sai về phản ứng của khách hàng đối với các vấn đề thiết kế trình bày của nhà phát triển. Thiếu quan tâm đến các đề nghị của khách hàng đề cập đến yêu cầu thay đổi và khách hàng trả lời cho các câu hỏi nêu ra bởi nhà phát triển trên một phần của nhà phát triển.

- Sự thiếu rõ ràng của các yêu cầu phần mềm:
  + Sử dụng môđun từ một dự án trước đó mà không phân tích đầy đủ về những thay đổi và thích nghi để phù hợp với yêu cầu mới.
  + Nhà phát triển quyết định bỏ qua một phần của các yêu cầu các chức năng.
  + Nhà phát triển bỏ qua các yêu cầu "nhỏ" và cuối cùng gây ra lỗi phần mềm
- Lỗi thiết kế logic:
  + Định nghĩa các yêu cầu phần mềm bằng các thuật toán sai.
  + Quy trình định nghĩa có chứa trình tự lỗi.
  + Sai sót trong các định nghĩa biên.
  + Thiếu sót trong các trạng thái hệ thống phần mềm được yêu cầu
  + Thiếu sót trong định nghĩa các hoạt động trái luật trong hệ thống phần mềm
- Lỗi coding: Một loạt các lý do các lập trình viên có thể gây ra các lỗi code: sự hiểu lầm các tài liệu thiết kế, sai sót trong ngôn ngữ lập trình, sai sót trong việc áp dụng CASE và các công cụ phát triển khác, sai sót trong lựa chọn dữ liệu…
- Không phù hợp với tài liệu và chỉ thị coding: Hầu hết các đơn vị phát triển có tài liệu và tiêu chuẩn để xác định nội dung, trình tự và định

dạng của văn bản, và code tạo ra bởi các thành viên. Để hỗ trợ yêu cầu này, đơn vị phát triển công khai các mẫu và hướng dẫn mã hóa. Các thành viên của nhóm phát triển, đơn vị được yêu cầu phải thực hiện theo các yêu cầu này.

- Thiếu sót trong quá trình kiểm thử:
    + Kế hoạch kiểm thử chưa hoàn chỉnh
    + Không hoàn chỉnh sửa chữa các lỗi được phát hiện (do sơ suất hay áp lực thời gian...)
- Lỗi thủ tục: Các thủ tục trực tiếp cho người sử dụng đối với các hoạt động là cần thiết ở mỗi bước của quá trình. Chúng có tầm quan trọng đặc biệt trong các hệ thống phần mềm, nơi các tiến trình được tiến hành nhiều bước, mỗi bước trong số đó có thể có nhiều kiểu dữ liệu và cho phép kiểm tra các kết quả trung gian
- Lỗi tài liệu: Các lỗi về tài liệu là vấn đề của các đội phát triển và bảo trì đều có sai sót trong tài liệu thiết kế và trong tài liệu hướng dẫn tích hợp trong thân của phần mềm. Những lỗi này có thể là nguyên nhân gây ra lỗi bổ sung trong giai đoạn phát triển tiếp và trong thời gian bảo trì.

1.2.3. Chất lượng phần mềm

Khái niệm chất lượng được hiểu là sự phù hợp với đặc điểm kỹ thuật trong sản xuất. Chất lượng được định nghĩa là vấn đề của sản phẩm và các dịch vụ có các đặc tính có thể đo lường thỏa mãn một đặc điểm kỹ thuật cố định - nghĩa là, sự phù hợp với một thông số kỹ thuật được xác định trước. Chất lượng được xác định độc lập với bất kỳ đặc điểm nào có thể đo lường được, nghĩa là chất lượng được định nghĩa là khả năng của sản phẩm hoặc dịch vụ để đáp ứng mong đợi của khách hàng - rõ ràng hay không.

Chất lượng sản phẩm phần mềm là khả năng đáp ứng toàn diện nhu cầu của người dùng về tính năng cũng như công dụng được nêu ra một cách tường minh trong những ngữ cảnh xác định. Trong một trường hợp cụ thể, có thể bao gồm các tính năng không tường minh. Có nhiều quan điểm và cách nhìn nhận chất lượng phần mềm.

Dưới quan điểm của người sử dụng, chất lượng phần mềm là mức độ làm thỏa mãn yêu cầu được đặc tả hoặc mong đợi đối về phần mềm, với thời gian và chi phí hợp lý:

+ Tính đúng đắn: đầy đủ, chính xác;

+ Tính tiện dụng: dễ học; dễ sử dụng; giao diện trực quan; tự nhiên;

+ Tính hiệu quả: tối ưu sử dụng thiết bị;

+ Tính tương thích trong nhập/xuất dữ liệu và tương tác;

+ Tính tiến hóa: một trong các tính chất quan trọng nhất được quan tâm xem xét trong ngành Công nghệ Phần mềm. Sự hoàn thiện dần và gắn liền với sự tiến hóa của phần cứng;

Dưới quan điểm của nhà phát triển, chất lượng phần mềm phản ánh các khía cạnh:

+ Dễ làm và dễ cập nhật: sử dụng lại, hợp chuẩn, tiếp nhận công nghệ mới, mềm dẻo;

+ An toàn.

Để có cái nhìn rõ ràng hơn về chất lượng phần mềm, người ta có thể dựa trên các định nghĩa về chất lượng phần mềm. Có rất nhiều định nghĩa về chất lượng phần mềm được đưa ra bởi các tổ chức, cá nhân khác nhau: IEEE (1991), Pressman (2000), v.v.

Định nghĩa 1.2: *Chất lượng phần mềm là mức độ mà một hệ thống, thành phần hệ thống hay tiến trình đáp ứng được yêu cầu đã được đặc tả.*

Định nghĩa 1.3: *Chất lượng phần mềm là mức độ mà một hệ thống, thành phần hệ thống hay tiến trình đáp ứng được yêu cầu và sự mong đợi của khách hàng hay người sử dụng.*

Thứ nhất, IEEE chỉ ra chất lượng phần mềm là một mức độ mà một hệ thống, thành phần hệ thống hay tiến trình đáp ứng được yêu cầu đã được đặc tả. Điều này dẫn tới sự phụ thuộc quá nhiều vào tài liệu đặc tả của yêu cầu. Điều này cho thấy, nếu việc xác định yêu cầu bị sai, thiếu thì một phần mềm được làm đúng với đặc tả chưa chắc đã là một phần mềm có chất lượng.

Thứ hai, IEEE chỉ ra chất lượng phần mềm là mức độ mà một hệ thống, thành phần hệ thống hay tiến trình đáp ứng được yêu cầu và sự mong đợi của khách hàng hay người sử dụng. Điều này có thể dẫn tới những yêu cầu vượt quá giới hạn có thể thực hiện được của phần mềm. Khách hàng đôi khi không có kiến thức về công nghệ, họ có thể đưa ra những mong muốn hết sức vô lý và có thể thay đổi yêu cầu với phần mềm nhiều lần, thậm chí thay đổi ngay trong giai đoạn cuối. Điều này gây nhiều khó khăn cho việc phát triển phần mềm.

Định nghĩa 1.4: *Sự phù hợp với các yêu cầu cụ thể về chức năng và hiệu suất, các tiêu chuẩn phát triển được ghi lại trong tài liệu với các đặc điểm được mong đợi ở tất cả các phần mềm được phát triển chuyên nghiệp.*

Theo Pressman (Định nghĩa 1.4), chất lượng phần mềm là sự phù hợp của các yêu cầu cụ thể về hiệu năng và chức năng, các tiêu chuẩn phát triển phần mềm được ghi lại rõ ràng bằng tài liệu với các đặc tính ngầm định của tất cả các phần mềm được phát triển chuyên nghiệp. Định nghĩa của Pressman đề xuất ba yêu cầu với chất lượng phần mềm phải được đáp ứng khi phát triển phần mềm:

- Các yêu cầu chức năng rõ ràng là nhân tố chính quyết định chất lượng đầu ra của phần mềm;

- Các tiêu chuẩn chất lượng phần mềm sẽ được nói đến trong hợp đồng;

- Các đặc tính ngầm định cần được đáp ứng trong quá trình phát triển cho dù không được nói đến rõ ràng trong hợp đồng.

Ngoài ra, còn một số khái niệm về chất lượng phần mềm:

- Là sự thỏa mãn yêu cầu (Crosby,1979), sản phẩm có đủ những đặc tính làm thỏa mãn khách hàng để tạo ra sự hài lòng về sản phẩm.

- Không có thiếu sót trong sản phẩm (Juran, 1988):

+ Mức độ mà một sản phẩm (hệ thống, một thành phần hay một xử lý) làm thỏa mãn cho các yêu cầu đã được định nghĩa;

+ Mức độ mà một sản phẩm làm thỏa mãn cho khách hàng, cho nhu cầu của người sử dụng hoặc cho các kỳ vọng về nó (IEEE, 1991).

1.2.4. Đảm bảo chất lượng phần mềm

Đảm bảo chất lượng bao gồm việc thiết lập một tập hợp các hoạt động có chủ đích và có hệ thống nhằm mang lại sự tin tưởng sẽ đạt được chất lượng của sản phẩm đòi hỏi.

Đảm bảo chất lượng phần mềm là đảm bảo dự án phần mềm sẽ hoàn thành đúng đặc tả, theo chuẩn mực định trước và các chức năng đòi hỏi, không có hỏng hóc và các vấn đề tiềm ẩn.

SQA bao gồm nhiệm vụ điều khiển và cải tiến tiến trình phát triển phần mềm ngay từ khi dự án bắt đầu. Nó có tác dụng "phòng ngừa" cái xấu, cái kém chất lượng.

Mục tiêu cuối cùng của đảm bảo chất lượng phần mềm là thỏa mãn khách hàng (costumer satisfaction) về: Thời gian; Ngân sách; Chất lượng.

Theo Daniel Galin [2], SQA là một tập hợp các nhiệm vụ/hoạt động QA được lên kế hoạch một cách có hệ thống trong các dự án phát triển/bảo trì sản phẩm phần mềm. Bản kế hoạch này đi kèm với quy trình phát triển phần mềm hay quy trình bảo trì phần mềm. Nó phù hợp với các yêu cầu kỹ thuật cũng như với

các yêu cầu quản lý và giúp cho dự án giữ được lịch biểu và các hoạt động trong phạm vi ngân sách dự án.

Yêu cầu chung của đảm bảo chất lượng phần mềm:

- Tuân thủ đặc tả là nền tảng để đo lường chất lượng.

- Các chuẩn (standards) được xác định trước dùng để phát triển các tiêu chí chất lượng và dẫn dắt quá trình công nghệ.

- Bên cạnh tuân thủ các yêu cầu tường minh (trong đặc tả), phần mềm phải tuân thủ các đặc tả không tường minh (dễ dùng, dễ bảo trì, tin cậy).

1.3. CÁC YẾU TỐ ẢNH HƯỞNG TỚI HOẠT ĐỘNG ĐẢM BẢO CHẤT LƯỢNG PHẦN MỀM

1.3.1. YẾU TỐ CON NGƯỜI

Yếu tố con người đóng vai trò quan trọng và quyết định đến chất lượng cuối cùng của sản phẩm phần mềm. Mặc dù phần mềm được phát triển dựa trên công nghệ và quy trình, nhưng yếu tố con người là nhân tố trung tâm ảnh hưởng đến hiệu quả của các phương pháp và công cụ kỹ thuật được sử dụng.

1.3.1.1. Đội dự án

Chất lượng phần mềm không chỉ phụ thuộc vào mã nguồn hay công nghệ được áp dụng mà còn chịu tác động trực tiếp từ kiến thức, kỹ năng, động lực làm việc và sự phối hợp của đội ngũ phát triển phần mềm. Một nhóm phát triển có kỹ năng cao, tinh thần trách nhiệm và khả năng làm việc nhóm tốt sẽ tạo ra sản phẩm phần mềm chất lượng cao. Ngược lại, nếu đội ngũ thiếu kỹ năng, thiếu sự phối hợp hoặc không được đào tạo đầy đủ, chất lượng phần mềm sẽ bị ảnh hưởng nghiêm trọng.

Một số vấn đề liên quan đến yếu tố đội dự án:

- Năng lực kỹ thuật của đội ngũ phát triển:
  + Kỹ năng lập trình: Đội ngũ phát triển cần thành thạo các ngôn ngữ

lập trình (Java, Python, C#,…) và framework (Spring, React, .NET…) được sử dụng trong dự án.

+ Kinh nghiệm của mỗi cá nhân: Đội ngũ có kinh nghiệm xử lý các vấn đề phức tạp sẽ giúp phát hiện và khắc phục lỗi nhanh chóng.
+ Kiến thức về kiểm thử phần mềm: Hiểu biết về các kỹ thuật kiểm thử (hộp trắng, hộp đen) và sử dụng công cụ kiểm thử tự động (Selenium, Jmeter,…).

- Động lực làm việc của đội ngũ phát triển:
  + Cam kết với chất lượng: Nhóm phát triển cần nhận thức rõ về tầm quan trọng của chất lượng phần mềm và sẵn sàng đầu tư thời gian để đảm bảo sản phẩm đạt yêu cầu.
  + Tinh thần làm việc nhóm: Sự phối hợp tốt giữa các thành viên trong nhóm phát triển (lập trình viên, tester, quản lý dự án) giúp phát hiện và khắc phục lỗi nhanh chóng.
  + Môi trường làm việc: Môi trường làm việc tích cực, khuyến khích sáng tạo và hỗ trợ phát triển kỹ năng sẽ giúp nhóm phát triển làm việc hiệu quả hơn.
- Đào tạo và phát triển năng lực nhân sự:
  + Đào tạo kỹ thuật: Định kỳ tổ chức các khóa học về kỹ thuật lập trình, kiểm thử, bảo mật phần mềm.
  + Đào tạo về quy trình và tiêu chuẩn chất lượng: Giúp đội ngũ hiểu rõ quy trình phát triển phần mềm và các tiêu chuẩn chất lượng (ISO/IEC 25010, IEEE 730).
  + Chia sẻ kiến thức và kinh nghiệm: Tổ chức các buổi trao đổi kiến thức, diễn đàn chất lượng phần mềm để đội ngũ có thể học hỏi lẫn nhau.
- Định hướng và quản lý đội ngũ phát triển

+ Quản lý năng suất làm việc: Đảm bảo rằng đội ngũ phát triển được giao đúng nhiệm vụ phù hợp với khả năng của họ.
+ Đánh giá hiệu suất: Định kỳ đánh giá năng lực và hiệu quả làm việc của từng thành viên trong nhóm.
+ Khuyến khích và khen thưởng: Áp dụng các chính sách khuyến khích, thưởng để động viên nhân viên khi đạt kết quả tốt trong công việc.

1.3.1.2. Thành viên ngoài

Thành viên tham gia bên ngoài có thể được phân chia thành 3 nhóm chính:

- Subcontractors: nhà thầu phụ, hiện nay được gọi là những tổ chức "outsourcing"
- Những nhà cung cấp phần mềm COTS (Commercial Off-The-Shelf Software) và những module phần mềm sử dụng lại.
- Khách hàng tham gia với tư cách người tham gia thực hiện dự án.

Rủi ro chủ yếu tới chất lượng dự án liên quan với thành viên ngoài trong cơ cấu của dự án là như sau:

- Ảnh hưởng tới thời gian hoàn thành của dự án.
- Chất lượng của các phần dự án được cung cấp bởi các thành viên bên ngoài.
- Giai đoạn bảo trì trong tương lai.
- Mất sự kiểm soát các bộ phận của dự án

1.3.2. Yếu tố dự án

Yếu tố dự án là các đặc điểm riêng của từng dự án phần mềm, bao gồm quy mô, độ phức tạp, phạm vi và yêu cầu của dự án. Các dự án lớn và phức tạp đòi hỏi các quy trình SQA nghiêm ngặt hơn, trong khi các dự án nhỏ có thể áp dụng quy trình SQA đơn giản hơn. Đặc điểm của dự án sẽ quyết định phương pháp kiểm thử, công cụ sử dụng và cách thức đánh giá chất lượng. Việc nhận diện và kiểm

soát các yếu tố này giúp tổ chức đảm bảo rằng quy trình SQA được tùy chỉnh phù hợp với từng dự án cụ thể.

Yếu tố dự án, bao gồm:

- Quy mô dự án: Dự án lớn yêu cầu nhiều hoạt động SQA hơn để đảm bảo chất lượng.
- Độ phức tạp kỹ thuật: Dự án phức tạp về mặt kỹ thuật đòi hỏi các biện pháp SQA kỹ lưỡng hơn.
- Phạm vi của các thành phần phần mềm tái sử dụng: Độ phổ biến của các thành phần tái sử dụng trong dự án có thể giảm bớt khối lượng công việc SQA.
- Mức độ nghiêm trọng của hậu quả nếu dự án thất bại: Nếu hậu quả của thất bại cao, cường độ của hoạt động SQA sẽ được gia tăng .

1.3.3. Yếu tố quy trình

Quy trình là nền tảng quan trọng giúp đảm bảo rằng phần mềm được phát triển theo các phương pháp khoa học, có kiểm soát và đạt được chất lượng cao. Quy trình phát triển phần mềm được thiết kế để xác định các bước thực hiện từ giai đoạn phân tích yêu cầu, thiết kế, lập trình, kiểm thử đến bảo trì. Đảm bảo chất lượng phần mềm không chỉ phụ thuộc vào kỹ năng của đội ngũ phát triển, mà còn dựa vào việc áp dụng các quy trình phù hợp và tuân thủ chúng một cách nghiêm túc.

Yếu tố quy trình bao gồm việc lựa chọn mô hình phát triển phần mềm phù hợp, thiết lập quy trình kiểm thử rõ ràng, quản lý cấu hình phần mềm để đảm bảo tính nhất quán của các thành phần, và kiểm soát các thay đổi phát sinh trong suốt vòng đời dự án. Đảm bảo rằng các quy trình này được áp dụng đúng cách sẽ giúp giảm thiểu rủi ro phát triển phần mềm và nâng cao chất lượng sản phẩm cuối cùng.

1.3.3.1. Quy trình phát triển phần mềm

Lựa chọn mô hình phát triển phần mềm phù hợp là yếu tố quan trọng để đảm bảo chất lượng phần mềm. Các mô hình phát triển giúp:

- Đảm bảo dự án được phát triển theo quy trình nhất quán và có kiểm soát,
- Xác định rõ các hoạt động đảm bảo chất lượng trong từng giai đoạn phát triển.
- Hỗ trợ xác định và quản lý rủi ro trong quá trình phát triển.

Các mô hình phát triển phần mềm phổ biến bao gồm:

- **Waterfall (Thác nước):** Mô hình truyền thống với các giai đoạn phát triển tuần tự: phân tích, thiết kế, lập trình, kiểm thử, triển khai. Waterfall phù hợp cho các dự án có yêu cầu rõ ràng và ít thay đổi.
- **Agile (Linh hoạt):** Mô hình phát triển lặp lại và gia tăng, trong đó phần mềm được phát triển thông qua các chu kỳ ngắn (Sprint). Mô hình linh hoạt phù hợp với các dự án có yêu cầu thay đổi liên tục.
- **V-Model (Mô hình chữ V):** Là phiên bản mở rộng của Waterfall, nhưng tích hợp kiểm thử vào từng giai đoạn phát triển. Đảm bảo rằng mọi yêu cầu và thiết kế đều được kiểm thử ngay từ đầu.
- **Incremental (Gia tăng):** Phần mềm được phát triển theo từng phần nhỏ, với mỗi phần là một phiên bản hoàn chỉnh có thể sử dụng được. Mô hình gia tăng phù hợp với các dự án phức tạp cần phát triển theo giai đoạn.
- **Spiral (Xoắn ốc):** Mô hình tập trung vào việc kiểm soát rủi ro với các vòng lặp phát triển bao gồm lập kế hoạch, phân tích rủi ro, phát triển và đánh giá.

1.3.3.2. Quy trình kiểm thử phần mềm

Quy trình kiểm thử phần mềm là yếu tố quan trọng giúp phát hiện và khắc phục lỗi trong phần mềm trước khi phát hành. Quy trình kiểm thử phần mềm nhằm:

- Đảm bảo rằng tất cả các chức năng của phần mềm đều được kiểm tra đầy đủ.
- Phát hiện và sửa lỗi trước khi phần mềm được phát hành.
- Đảm bảo rằng phần mềm đáp ứng yêu cầu của khách hàng.

Một quy trình kiểm thử hiệu quả bao gồm các giai đoạn kiểm thử rõ ràng, được thiết kế để đảm bảo rằng tất cả các chức năng của phần mềm đều được kiểm tra đầy đủ:

- **Kiểm thử đơn vị (Unit Testing):** Đánh giá tính đúng đắn của từng thành phần (hàm, lớp) trong mã nguồn.
- **Kiểm thử tích hợp (Integration Testing):** Đánh giá khả năng tương tác giữa các thành phần phần mềm khi được tích hợp với nhau.
- **Kiểm thử hệ thống (System Testing):** Đánh giá tính đầy đủ và tính đúng đắn của toàn bộ hệ thống phần mềm.
- **Kiểm thử nghiệm thu (User Acceptance Testing - UAT):** Đánh giá tính phù hợp của phần mềm với yêu cầu của người dùng cuối.

1.3.3.3. Quy trình quản lý cấu hình phần mềm

Quản lý cấu hình phần mềm là quá trình kiểm soát và theo dõi các phiên bản của tất cả các thành phần phần mềm trong dự án. Mục tiêu của quản lý cấu hình là đảm bảo rằng mọi thay đổi trong mã nguồn, tài liệu và các thành phần phần mềm khác đều được ghi nhận và kiểm soát chặt chẽ:

- Đảm bảo tính nhất quán của các thành phần phần mềm trong toàn bộ dự án.
- Hỗ trợ theo dõi và khắc phục lỗi một cách hiệu quả.
- Đảm bảo rằng mọi thay đổi đều được ghi lại và có thể theo dõi.

Các nội dung:

- **Quản lý phiên bản mã nguồn:** Sử dụng hệ thống quản lý mã nguồn (Git, SVN,…) để lưu trữ và theo dõi các phiên bản mã nguồn.
- **Quản lý tài liệu:** Đảm bảo rằng các tài liệu dự án (SRS, SDD, Test plan,…) được cập nhật và kiểm soát phiên bản.
- **Quản lý môi trường phát triển:** Đảm bảo rằng môi trường phát triển (IDE, thư viện, công cụ) được cấu hình nhất quán.
- **Kiểm soát thay đổi:** Đảm bảo rằng mọi thay đổi trong mã nguồn hoặc tài liệu đều được đánh giá, phê duyệt và theo dõi.

1.3.3.4. Quy trình quản lý thay đổi

Trong suốt vòng đời phát triển phần mềm, các yêu cầu thay đổi là không thể tránh khỏi. Quy trình quản lý thay đổi (Change Management) đảm bảo rằng mọi yêu cầu thay đổi đều được xem xét, đánh giá, phê duyệt và theo dõi chặt chẽ:

- **Đề xuất thay đổi:** Người dùng hoặc nhóm phát triển đưa ra yêu cầu thay đổi.
- **Đánh giá thay đổi:** Nhóm quản lý dự án đánh giá tác động của thay đổi đối với chất lượng, tiến độ và chi phí.
- **Phê duyệt thay đổi:** Quyết định có thực hiện thay đổi hay không, dựa trên kết quả đánh giá.
- **Thực hiện thay đổi:** Thay đổi được triển khai và kiểm thử để đảm bảo không ảnh hưởng đến các tính năng hiện tại.
- **Theo dõi và báo cáo thay đổi:** Lưu lại lịch sử thay đổi và báo cáo kết quả kiểm thử thay đổi.

1.3.4. Yếu tố công nghệ

Trong đảm bảo chất lượng phần mềm, yếu tố công nghệ đóng vai trò quan trọng trong việc xác định khả năng phát triển, kiểm thử, triển khai và bảo trì phần mềm. Công nghệ không chỉ bao gồm các công cụ và môi trường phát triển mà còn liên quan đến các công nghệ nền tảng, khả năng tương thích với các hệ thống khác

và khả năng bảo mật của phần mềm. Một hệ thống phần mềm được xây dựng trên nền tảng công nghệ tiên tiến và được kiểm soát chặt chẽ về bảo mật sẽ có khả năng đảm bảo chất lượng cao hơn, mang lại hiệu suất tốt và an toàn cho người dùng. Yếu tố công nghệ trong SQA không chỉ ảnh hưởng đến hiệu quả phát triển phần mềm mà còn ảnh hưởng đến chất lượng sản phẩm cuối cùng.

Việc lựa chọn công nghệ phù hợp và áp dụng đúng cách là yếu tố quyết định đến khả năng mở rộng, hiệu suất và độ bảo mật của phần mềm. Tác động của yếu tố công nghệ đến chất lượng phần mềm:

- Tăng hiệu quả phát triển: Công cụ tự động hóa và môi trường phát triển tích hợp giúp giảm thiểu lỗi và tăng tốc độ phát triển.
- Đảm bảo tính nhất quán: Công cụ quản lý cấu hình giúp theo dõi và kiểm soát các phiên bản phần mềm.
- Tăng cường bảo mật: Công cụ kiểm thử bảo mật giúp phát hiện và xử lý các lỗ hổng bảo mật.
- Hỗ trợ quản lý lỗi: Công cụ theo dõi lỗi giúp đảm bảo mọi lỗi được ghi nhận và xử lý đúng cách.
- Đảm bảo tính dễ bảo trì: Công nghệ quản lý mã nguồn và cấu hình giúp phần mềm dễ dàng bảo trì và cập nhật.

1.3.4.1. Môi trường phát triển

- Môi trường phát triển tích hợp (IDE - Integrated Development Environment). Cung cấp không gian lập trình, gỡ lỗi và kiểm thử tập trung cho lập trình viên: Visual Studio, Eclipse, IntelliJ IDEA),…
- Công cụ quản lý mã nguồn (Version control tools: Git (GitHub, GitLab), SVN (Apache subversion)):
  + Đảm bảo rằng tất cả mã nguồn trong dự án được kiểm soát phiên bản và lưu trữ an toàn.

+ Hỗ trợ theo dõi lịch sử thay đổi mã nguồn và khôi phục phiên bản.

- Công cụ kiểm thử tự động (Automated testing tools: Selenium (kiểm thử giao diện), JMeter (kiểm thử hiệu suất)): Tự động hóa quy trình kiểm thử, giúp tiết kiệm thời gian và giảm thiểu lỗi do con người.
- Công cụ theo dõi lỗi (Bug tracking tools: JIRA, Bugzilla, Redmine):
  + Đảm bảo rằng mọi lỗi phát hiện trong quá trình phát triển và kiểm thử đều được ghi nhận, theo dõi và xử lý.
  + Hỗ trợ phân loại lỗi theo mức độ nghiêm trọng: Critical, High, Medium, Low.

1.3.4.2. Công nghệ phát triển và quản lý phần mềm

- Ngôn ngữ lập trình và thư viện hỗ trợ: Lựa chọn ngôn ngữ lập trình phù hợp giúp tăng tốc độ phát triển và giảm thiểu lỗi. Ví dụ: Python (đơn giản và linh hoạt), Java (bảo mật và đa nền tảng), JavaScript (phát triển giao diện người dùng).

- Frameworks và API hỗ trợ: Giúp tiêu chuẩn hóa quá trình phát triển và tăng khả năng tái sử dụng. Ví dụ: Spring Boot (Java), Django (Python), React (JavaScript).

- Công cụ quản lý cấu hình phần mềm SCM (Software Configuration Management: Git (quản lý mã nguồn), Docker (quản lý môi trường triển khai)):
  + Đảm bảo tính nhất quán của các thành phần phần mềm trong toàn bộ dự án.
  + Hỗ trợ theo dõi và khắc phục lỗi một cách hiệu quả.
  + Đảm bảo mọi thay đổi trong mã nguồn, tài liệu và cấu hình hệ thống đều được theo dõi và kiểm soát.

- Quản lý thay đổi (Change management: JIRA, Azure boards): Đảm bảo rằng mọi yêu cầu thay đổi đều được xem xét, đánh giá, phê duyệt và theo dõi chặt chẽ

- Công cụ bảo mật phần mềm (Security tools: OWASP ZAP (kiểm thử bảo mật ứng dụng web), Nessus (quản lý lỗ hổng)): Đảm bảo rằng phần mềm được bảo vệ trước các mối đe dọa bảo mật.
- Mã hóa và bảo vệ dữ liệu (Encryption and data protection): Đảm bảo rằng dữ liệu người dùng được bảo vệ khỏi các truy cập trái phép. Công nghệ: SSL/TLS (mã hóa kết nối), AES (Advanced Encryption Standard).
- Công cụ hỗ trợ đánh giá và đo lường:
    + Selenium: Đánh giá tính năng chức năng của ứng dụng web.
    + JMeter: Đánh giá hiệu suất và khả năng chịu tải của hệ thống.
    + SonarQube: Đánh giá chất lượng mã nguồn (Code quality) dựa trên các tiêu chí như độ phức tạp, bảo mật và khả năng bảo trì.
    + Burp suite: Đánh giá bảo mật của ứng dụng web, phát hiện các lỗ hổng bảo mật.
    + JIRA: Theo dõi và quản lý các lỗi phát hiện trong quá trình kiểm thử.

1.3.5. Yếu tố đánh giá và đo lường

Yếu tố đánh giá và đo lường đóng vai trò quan trọng trong việc xác định mức độ chất lượng của phần mềm và đánh giá hiệu quả của quy trình phát triển. Đánh giá và đo lường không chỉ giúp xác định các khía cạnh chất lượng hiện tại của phần mềm mà còn cung cấp thông tin quan trọng để cải tiến quy trình, giảm thiểu rủi ro và nâng cao chất lượng sản phẩm trong tương lai.

Đánh giá và đo lường chất lượng phần mềm không chỉ giới hạn ở việc kiểm thử mà còn bao gồm việc sử dụng các tiêu chuẩn chất lượng, đo lường hiệu suất hệ thống, theo dõi lỗi và đánh giá độ tin cậy của phần mềm. Việc thiết lập các tiêu chí đo lường rõ ràng và áp dụng các phương pháp đánh giá hiệu quả sẽ giúp tổ chức theo dõi và nâng cao chất lượng phần mềm một cách có hệ thống.

1.3.5.1. Thiết lập tiêu chuẩn đánh giá chất lượng

Lựa chọn và thiết lập tiêu chuẩn chất lượng của dự án:

- Tiêu chuẩn ISO/IEC 25010: Đánh giá chất lượng sản phẩm phần mềm dựa trên các đặc tính như tính phù hợp chức năng, hiệu suất, khả năng tương thích, khả năng sử dụng, độ tin cậy, bảo mật, khả năng bảo trì và khả năng di chuyển.
- ISO 9000-3: Đánh giá hệ thống quản lý chất lượng phần mềm, bao gồm việc tuân thủ quy trình phát triển và kiểm thử phần mềm.
- IEEE Std 730-2014: Đánh giá kế hoạch đảm bảo chất lượng phần mềm (SQA Plan) và các hoạt động đảm bảo chất lượng.
- CMMI (Capability Maturity Model Integration): Đánh giá mức độ trưởng thành của quy trình phát triển phần mềm trong tổ chức.

1.3.5.2. Xác định độ đo

Độ đo chất lượng phần mềm là các chỉ số định lượng giúp đánh giá và đo lường các khía cạnh chất lượng của phần mềm. Các độ đo phổ biến bao gồm:

- Độ đo hiệu quả kiểm thử:
  + Tỷ lệ test case pass/fail: Đánh giá tỷ lệ thành công và thất bại của các ca kiểm thử.
  + Tỷ lệ phát hiện lỗi: Đánh giá khả năng phát hiện lỗi trong quá trình kiểm thử.
- Độ đo hiệu suất hệ thống:
  + Thời gian phản hồi trung bình (Average Response Time):
  + Số lượng người dùng đồng thời: Đo lường khả năng hệ thống xử lý số lượng người dùng truy cập đồng thời.
- Độ đo bảo mật:
  + Số lượng lỗ hổng bảo mật phát hiện được: Đánh giá mức độ an toàn của phần mềm.

+ Tỷ lệ khắc phục lỗ hổng bảo mật: Đánh giá hiệu quả xử lý các lỗ hổng bảo mật.

- Độ đo độ tin cậy:
  + Tỷ lệ uptime của hệ thống: Đánh giá độ ổn định của hệ thống.
  + Số lỗi nghiêm trọng trong quá trình vận hành: Đánh giá mức độ ảnh hưởng của lỗi đối với người dùng.
- Độ đo bảo trì: Thời gian trung bình sửa lỗi (MTTR - Mean Time to Repair): Đo lường thời gian trung bình để sửa một lỗi.
- Tỷ lệ lỗi tái phát: Đo lường tỷ lệ lỗi bị lặp lại sau khi đã được sửa.

1.3.5.3. Phương pháp đánh giá chất lượng phần mềm

Lựa chọn phương pháp đánh giá cũng ảnh hưởng tới SQA. Các phương pháp đánh giá:

- Đánh giá nội bộ (Internal Audit): Được thực hiện bởi đội ngũ SQA của tổ chức để đánh giá tuân thủ quy trình và tiêu chuẩn chất lượng.
- Đánh giá bên ngoài (External Audit): Được thực hiện bởi tổ chức chứng nhận để xác nhận rằng hệ thống quản lý chất lượng của tổ chức đáp ứng tiêu chuẩn quốc tế.
- Đánh giá từ khách hàng (Customer Evaluation): Được khách hàng thực hiện để đảm bảo rằng sản phẩm phần mềm đáp ứng các yêu cầu đã đề ra.

1.3.6. Yếu tố quản lý

Yếu tố quản lý tập trung vào việc lập kế hoạch, tổ chức, điều phối và kiểm soát các hoạt động đảm bảo chất lượng phần mềm. Nó bao gồm việc thiết lập mục tiêu chất lượng, quản lý tiến độ dự án, phân bổ nguồn lực, quản lý chi phí và rủi ro. Nếu việc quản lý chất lượng được thực hiện hiệu quả, dự án sẽ có khả năng phát hiện và khắc phục lỗi sớm, giảm thiểu chi phí và nâng cao sự hài lòng của khách hàng. Ngược lại, nếu quản lý yếu kém, dự án sẽ dễ bị trễ tiến độ, vượt ngân

sách và không đảm bảo chất lượng sản phẩm. Các yếu tố quản lý ảnh hưởng tới SQA:

- Quản lý tiến độ dự án: Đảm bảo dự án được hoàn thành đúng thời hạn.
- Quản lý chi phí: Đảm bảo ngân sách cho các hoạt động đảm bảo chất lượng phần mềm được phân bổ hợp lý.
- Quản lý rủi ro: Xác định, đánh giá và xử lý các rủi ro chất lượng có thể xảy ra trong dự án.
- Phân công trách nhiệm: Định nghĩa rõ vai trò và trách nhiệm của từng thành viên trong đội SQA.

1.4. CÁC THÀNH PHẦN ĐẢM BẢO CHẤT LƯỢNG PHẦN MỀM

1.4.1. Công cụ hỗ trợ đảm bảo chất lượng phần mềm

1.4.1.1. Thủ tục và chỉ dẫn

Thủ tục, là các hoạt động hoặc quá trình chi tiết được thực hiện theo một phương pháp nhất định nhằm mục đích hoàn thành một nhiệm vụ.

Các thủ tục được một tổ chức thông qua được coi là ràng buộc đối với nhân viên của tổ chức đó, có nghĩa là mỗi nhân viên phải thực hiện nhiệm vụ của mình theo các bước xuất hiện trong tài liệu thủ tục liên quan, thường mang tên của nhiệm vụ được chỉ định.

Các thủ tục cũng có xu hướng phổ biến trong tổ chức, nghĩa là chúng được áp dụng bất cứ khi nào nhiệm vụ được thực hiện, bất kể người thực hiện nhiệm vụ hay bối cảnh tổ chức.

Có thể coi thủ tục:

- Là những hoạt động và tiến trình chi tiết được thực hiện theo một phương thức đã đưa ra với mục đích hoàn thiện một một nhiệm vụ;
- Là các ràng buộc đối với nhân viên của tổ chức;
- Được áp dụng mỗi khi nhiệm vụ được thực hiện.

Chỉ dẫn giúp cho một nhóm hoặc một bộ phận hoặc toàn bộ tổ chức thực hiện công việc đúng và chính xác. Như vậy, mục đích của Chỉ thị là chất lượng;

Chỉ dẫn còn gọi là hướng dẫn công việc hay bổ sung các thủ tục bằng cách cung cấp các chi tiết rõ ràng chỉ phù hợp với nhu cầu của một nhóm, bộ phận hoặc đơn vị.

Sự cần thiết của thủ tục và chỉ thị:

- Các tổ chức đảm bảo chất lượng phần mềm đặc biệt quan tâm đến các thủ tục và chỉ thị công việc SQA, bởi chúng có ảnh hưởng đến chất lượng của sản phẩm phần mềm, bảo trì phần mềm vào quản lý dự án.

- Các thủ tục SQA đòi hỏi phải phù hợp với chính sách chất lượng của tổ chức nhưng phải phù hợp tới các chuẩn SQA quốc gia và quốc tế.

- Chuẩn ISO 9000-3 (ISO, 1997; ISO/IEC, 2001) là một trong những chuẩn chứng nhận chính về thủ tục.

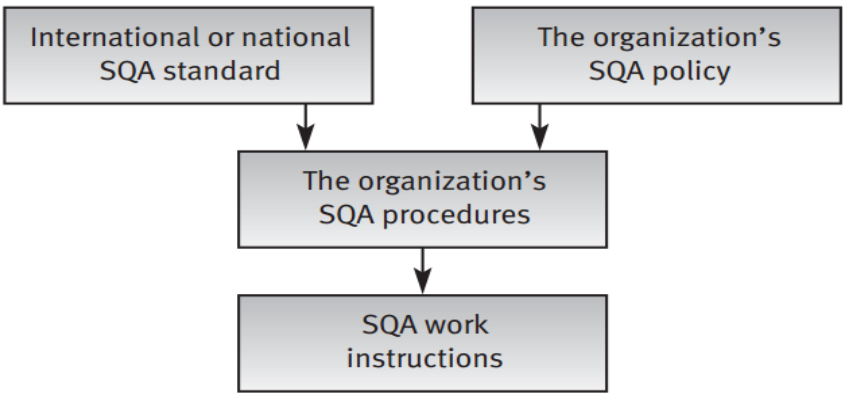
Mục đích của thủ tục và chỉ dẫn:

- Thực hiện công việc, tiến trình hoặc các hoạt động theo cách có hiệu quả nhất mà không sai lệch yêu cầu chất lượng.

- Thống nhất trong quá trình thực hiện, đạt được bởi tuân theo các thủ tục và chỉ thị công việc, sẽ giảm thiểu sự hiểu lầm dẫn đến lỗi phần mềm.

- Đơn giản hóa sự phối hợp giữa các công việc và hoạt động được thực hiện bởi các bộ phận của tổ chức. Phối hợp tốt nghĩa là ít lỗi hơn

Hình 2.1 biểu diễn phân cấp phát triển thủ tục và hướng dẫn công việc trong tổ chức đảm bảo chất lượng phần mềm



Hình 2.1 Biểu đồ phân cấp phát triển thủ tục và hướng dẫn công việc

1.4.1.2. Mẫu và danh mục kiểm tra

Mẫu (Template) được sử dụng như một sự chỉ dẫn định dạng của một thứ gì đó sẽ được tạo ra. Nhưng khi được áp dụng trong công nghệ phần mềm, thuật ngữ template ám chỉ một định dạng (đặc biệt là các bảng khái niệm) được tạo ra bởi các tổ chức, được áp dụng khi soạn thảo một bản báo cáo hoặc viết tài liệu.

Việc áp dụng các template có thể là bắt buộc (đối với một vài tài liệu) hoặc không bắt buộc (đối với các tài liệu khác).

Một số mẫu được định nghĩa trong dự án:

- Kế hoạch kiểm tra phần mềm (STP);

- Mô tả kiểm tra phần mềm (STD);

- Báo cáo kiểm tra phần mềm (STR);

- Yêu cầu thay đổi phần mềm (SCR);

- Tài liệu về cấu hình phần mềm.

Sử dụng template tạo nhiều thuận lợi cho các đội phát triển và đội rà soát. Đối với đội phát triển, sử dụng khuôn hình có các ưu điểm:

- Hỗ trợ việc chuẩn bị tài liệu.

- Đảm bảo các tài liệu được chuẩn bị bởi các nhà phát triển đầy đủ hơn bởi tất cả các chủ đề trong tài liệu đã được định nghĩa và được xem xét lại nhiều lần

bởi các chuyên gia thông qua việc sử dụng khuôn hình. Các lỗi thông thường, như là bỏ qua một chủ đề thì khó xảy ra hơn.

- Dễ dàng tích hợp các thành viên mới trong đội thông qua sự quen thuộc. Cấu trúc tiêu chuẩn của tài liệu, được chuẩn bị theo các khuôn hình mà có thể các thành viên mới đã biết từ trước khi làm việc trong các đội hoặc tổ chức khác, làm cho việc tìm kiếm thông tin dễ dàng hơn. Nó cũng giải quyết ổn thỏa việc chuẩn bị tài liệu tiếp theo khi mà các phần tài liệu đã được chuẩn bị bởi thành viên trong đội khác có thể rời đi hoặc không.

- Dễ dang xem xét lại tài liệu bằng cách loại bỏ sự cần thiết nghiên cứu cấu trúc của một tài liệu và xác nhận sự đầy đủ của nó, khi mà tài liệu được dựa trên khuôn hình thích hợp. Nó cũng làm đơn giản việc xem xét lại tài liều đầy đủ vì cấu trúc của nó là chuẩn và những người xem xét lại đã quen với nội dung trong đó (các chương, đoạn và phụ lục). Kết quả của tính nhất quán này là việc xem xét lại được cho là kỹ lưỡng và tốn ít thời gian hơn.

- Đối với các đội bảo trì phần mềm, việc sử dụng khuôn hình có ưu điểm: Làm cho dễ dàng xác định thông tin được yêu cầu cho các nhiệm vụ thực thi bảo trì.

Danh mục kiểm tra (Checklists) là một danh sách các việc cần hoàn thành (kế hoạch toàn diện) trước khi thực hiện một hoạt động. Mục đích của danh mục kiểm tra là cung cấp danh sách đầy đủ các đầu mục công việc (nội dung cần thực hiện) và không gian cho việc tìm kiếm tư liệu cần thiết cho dự án.

Vai trò của danh mục kiểm tra trong SQA:

- Giúp chuẩn bị và tự kiểm tra các tài liệu hoặc code cho nhiệm vụ/công việc trước khi thực hiện;

- Đảm bảo đầy đủ các tài liệu cho công tác rà soát và cải thiện hiệu quả của các phiên rà soát

1.4.2. Kiểm soát tiến độ

1.4.2.1. Vai trò của kiểm soát tiến độ dự án đến SQA

Hoạt động kiểm soát tiến độ của dự án có mục tiêu phát hiện sớm các vấn đề không bình thường. Nó giúp bắt đầu sớm thực hiện nhiệm vụ giải quyết vấn đề xảy ra. Vượt quá kỳ hạn các giai đoạn của dự án dẫn tới không đảm bảo ngân sách theo kế hoạch.

Sự chậm trễ trong việc hoàn thành các giai đoạn của dự án và vượt quá ngân sách nhiều lần là một trong các thất bại của ban quản lý dự án. Các tình huống dẫn tới điều này có thể do:

(i) Lập kế hoạch và ước tính ngân sách lạc quan quá mức hoặc thậm chí mù quáng (thường bắt đầu sớm hơn, trong giai đoạn phát triển đề xuất);

(ii) Quản lý rủi ro phần mềm không chuyên nghiệp và phản ứng chậm trễ hoặc không phù hợp với các rủi ro phần mềm.

(iii) Xác định chậm các khó khăn về lịch trình và ngân sách và đánh giá thấp mức độ của chúng.

Các tình huống *(i)* có thể được ngăn chặn bằng cách sử dụng các công cụ rà soát hợp đồng và lập kế hoạch dự án. *Kiểm soát tiến độ dự án* có khả năng ngăn ngừa các tình huống thuộc loại thứ (*ii*) và (*iii*).

Kiểm soát tiến độ dự án chủ yếu đề cập đến các khía cạnh quản lý của dự án, đó là lập kế hoạch, nhân lực và các nguồn lực khác, ngân sách và quản lý rủi ro.

Vị trí của hoạt động *kiểm soát tiến độ dự án* được chứng thực trong các tiêu chuẩn vòng đời SQA được đề cập đến trong các tài liệu: ISO 9000-3 (ISO (1997) Phần 4.9 và ISO / IEC (2001) Phần 7.5); IEEE / EIA Std 12207 (IEEE / ĐTM 1996, 1997a, 1997b; IEEE Std 1058 (IEEE, 1998a); IEEE Std 1490 (IEEE, 1998b)); CMM/CMMI.

Kiểm soát tiến độ dự án (trong CMM sử dụng thuật ngữ “theo dõi dự án phần mềm”) nhằm mục tiêu phát hiện sớm các sự kiện bất thường. Thúc đẩy việc

bắt đầu kịp thời hành động khắc phục và các phản ứng giải quyết vấn đề khi phát hiện các sự kiện bất thường.

1.4.2.2. Các hoạt động kiểm soát

Các thành phần chính của kiểm soát tiến độ dự án:

- Kiểm soát các hoạt động quản lý rủi ro: Điều này đề cập đến các hạng mục rủi ro phát triển phần mềm được xác định trong giai đoạn tiền dự án, những hạng mục được liệt kê trong tài liệu xem xét hợp đồng và kế hoạch dự án, cùng với các hạng mục rủi ro khác được xác định trong suốt tiến trình của dự án. Nhóm phát triển phần mềm đối phó với các hạng mục rủi ro phần mềm bằng cách áp dụng các hoạt động quản lý rủi ro có hệ thống. Việc kiểm soát tiến độ của quản lý rủi ro bắt đầu bằng việc chuẩn bị các bản đánh giá định kỳ về trạng thái của các hạng mục rủi ro phần mềm và kết quả mong đợi của các hoạt động quản lý rủi ro được thực hiện sau khi chúng thực hiện. Dựa trên các báo cáo này, các nhà quản lý dự án được kỳ vọng sẽ can thiệp và giúp đưa ra giải pháp trong những trường hợp khắc nghiệt hơn. Công cụ hỗ trợ kiểm soát hoạt động quản lý rủi ro cho dự án:

+ Danh sách danh mục các rủi ro (bao gồm mốc thời gian), giải pháp dự kiến.

+ Danh sách các trường hợp ngoại lệ của các hạng mục rủi ro phần mềm (bao gồm mốc thời gian).

- Kiểm soát lịch trình dự án: Hoạt động này đề cập đến việc tuân thủ của dự án với thời gian biểu đã được phê duyệt và trong hợp đồng đã được kỹ kết. Việc theo dõi lịch trình chủ yếu dựa trên các mốc quan trọng để xác định sự chậm trễ trong việc hoàn thành các hoạt động theo kế hoạch phát triển phần mềm. Các mốc quan trọng đặt ra trong hợp đồng bao gồm ngày giao các sản phẩm phần mềm cụ thể cho khách hàng hoặc hoàn thành một hoạt động phát triển, thường được chú trọng đặc biệt. Ban lãnh đạo và người chịu trách nhiệm sẽ tập trung các hoạt động kiểm soát của mình vào những sự chậm trễ nghiêm trọng, điều này có thể ảnh hưởng đáng kể đến thời hạn hoàn thành cuối cùng của dự án. Phần lớn thông tin

cần thiết cho việc kiểm soát tiến độ dự án của ban quản lý có trên các báo cáo quan trọng và các báo cáo định kỳ khác. Để đáp ứng thông tin này, ban quản lý có thể can thiệp bằng cách phân bổ các nguồn lực bổ sung hoặc thậm chí thương lượng lại lịch trình với khách hàng. Công cụ hỗ trợ kiểm soát lịch trình dự án:

+ Danh sách các hoạt động/nhiệm vụ có thể hoặc đã bị trì hoãn

+ Danh sách các hoạt động/nhiệm vụ quan trọng bị chậm tiến độ(sự chậm trễ, nếu không được sửa chữa sẽ ảnh hưởng đến ngày hoàn thành dự án)

+ Cập nhật hoạt bị chậm trễ đi kèm báo cáo tiến độ và các biện pháp hiệu chỉnh được áp dụng (cho các nhóm, đơn vị phát triển, v.v.)

+ Danh sách các mốc thời gian theo kế hoạch bị chậm hoặc chưa bắt đầu.

+ Cập nhật các mốc thời gian bị trì hoãn đi kèm báo cáo tiến độ và các biện pháp hiệu chỉnh đã áp dụng.

- Kiểm soát tài nguyên dự án: Hoạt động này tập trung vào sự kiểm soát của ban quản lý dựa trên các báo cáo định kỳ về việc sử dụng tài nguyên và so sánh việc sử dụng thực tế với việc sử dụng theo lịch trình. Mức độ sai lệch trong việc sử dụng tài nguyên chỉ có thể được đánh giá từ quan điểm về tiến độ của dự án. Nếu những sai lệch là chính đáng, ban quản lý có thể can thiệp bằng cách tăng các nguồn lực được phân bổ; cách khác, ban quản lý có thể thay đổi nguồn lực bằng cách tổ chức lại các nhóm dự án, sửa đổi kế hoạch của dự án, v.v. Công cụ hỗ trợ kiểm soát tài nguyên của dự án:

+ Kế hoạch cấp phát tài nguyên cho các hoạt động/giai đoạn thực hiện của dự án; cấp cho các nhóm/đơn vị phát triển, vv…

+ Vấn đề sử dụng tài nguyên dự của án theo tình thực tế trong quá trình thực hiện dự án;

+ Vấn đề sử dụng tài nguyên dự án tính từ khi bắt đầu thực hiện dự án đến từng thời điểm trong dự án;

+ Cập nhật kế hoạch phân bổ lại nguồn tài nguyên dự án theo các phát sinh trong quá trình thực hiện dự án.

- Kiểm soát ngân sách dự án: Hoạt động này dựa trên sự so sánh giữa các khoản chi tiêu trong thực tế với lịch trình. Các mục ngân sách chính cần kiểm soát là: Nguồn nhân lực; Các cơ phục vụ phát triển dự án; Mua phần mềm COTS (Commercial off the shelf); Mua phần cứng; Thanh toán cho nhà thầu phụ. Kiểm soát ngân sách là hoạt động được ưu tiên hàng đầu của ban quản lý vì nó ảnh hưởng trực tiếp đến khả năng sinh lời của dự án. Công cụ hỗ trợ kiểm soát ngân sách của dự án:

+ Kế hoạch tài chính cho các hoạt động/nhiệm vụ, cho các nhóm/đơn vị phát triển, vv…

+ Báo cáo sử dụng tài chính chi tiết theo từng giai đoạn thực hiện dự án và tổng kinh phí từ khi bắt đầu thực hiện dự án;

+ Báo cáo sự chênh lệch giữa kế hoạch phân bổ và thực tế trong việc sử dụng tài chính cho các giai đoạn thực hiện dự án hoặc tích lũy từ khi bắt đầu thực hiện dự án.

+ Cập nhật các phát sinh tài chính trong kế hoạch tài chính của dự án.

1.4.2.3. Phân bổ trách nhiệm thực hiện các nhiệm vụ kiểm soát

Kiểm soát tiến độ dự án thường dựa trên việc phân bổ trách nhiệm thực hiện các nhiệm vụ kiểm soát quá trình phù hợp với đặc điểm của dự án và quản lý tiến độ dự án. Việc phân bổ trách nhiệm thực hiện các nhiệm vụ kiểm soát quá trình phù hợp với đặc điểm của dự án, bao gồm:

- Cá nhân/đơn vị quản lý chịu trách nhiệm thực hiện nhiệm vụ kiểm soát tiến độ;

- Tần suất báo cáo theo yêu cầu từ mỗi đơn vị của dự;

- Trưởng dự án báo cáo ngay với bộ phận quản lý liên quan khi có tình huống phát sinh;

- Cấp dưới báo cáo ngay cho cấp trên khi có tình huống phát sinh.

1.5. CÂU HỎI VÀ BÀI TẬP

1.5.1. Câu hỏi

1. Phần mềm được hiểu như thế nào dưới góc nhìn của người sử dụng và chuyên viên Tin học?
2. Các thành phần nào cấu thành nên phần mềm theo định nghĩa của IEEE?
3. Vì sao người ta gọi quá trình sản xuất phần mềm là "phát triển phần mềm"?
4. Ba giai đoạn chính của vòng đời phát triển phần mềm (SDLC) là gì?
5. Những đặc điểm nào khiến phần mềm khác biệt so với sản phẩm công nghiệp truyền thống?
6. Thách thức lớn nhất trong việc đảm bảo chất lượng phần mềm là gì?
7. Làm thế nào để phân biệt giữa Error, Fault và Failure trong phát triển phần mềm?
8. Có bao nhiêu nguyên nhân phổ biến gây ra lỗi phần mềm và những nguyên nhân đó là gì?
9. Chất lượng phần mềm được nhìn nhận khác nhau như thế nào từ góc độ người dùng và nhà phát triển?
10. Mục tiêu cuối cùng của hoạt động đảm bảo chất lượng phần mềm là gì?

1.5.2. Bài tập

1. Phân tích và so sánh điểm giống và khác nhau giữa phần mềm và các sản phẩm công nghiệp khác về đặc điểm sản phẩm và quá trình phát triển.
2. Vẽ sơ đồ vòng đời phát triển phần mềm (SDLC) với các giai đoạn chính và liệt kê hoạt động tương ứng cho mỗi giai đoạn.
3. Thảo luận nhóm lựa chọn một trong các mô hình phát triển phần mềm (Waterfall, Scrum, Spiral...) và phân tích ưu – nhược điểm của nó trong đảm bảo chất lượng phần mềm.
4. Cho ví dụ thực tế về một lỗi phần mềm dẫn đến sự cố lớn. Phân tích lỗi

thuộc loại Error, Fault hay Failure?

5. Thảo luận nhóm và cho biết, tại sao phần mềm khó đảm bảo chất lượng hơn các sản phẩm công nghiệp khác? Làm thế nào để khắc phục?
6. Chọn một hệ thống phần mềm thực tế (ví dụ: hệ thống thư viện, bán hàng, ngân hàng) và phân tích các yếu tố ảnh hưởng đến chất lượng phần mềm trong hệ thống đó.
7. Lập bảng tổng hợp các nguyên nhân gây lỗi phần mềm (theo nhóm nguyên nhân) và biện pháp phòng tránh tương ứng.